

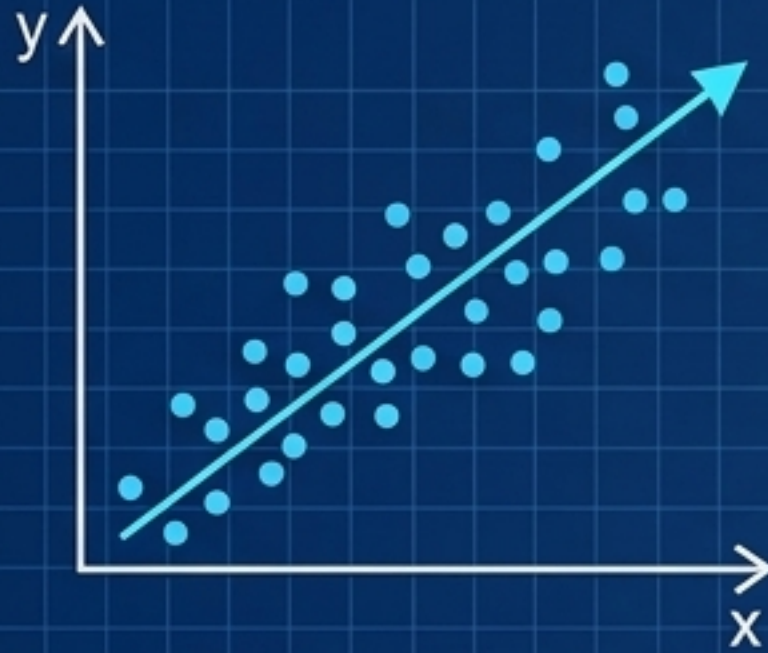
Machine Learning & KI-Algorithmen: Von der Theorie zur Business-Praxis

Ein Architekturplan für Modellierung,
Evaluation und MLOps.



Die drei Architektur-Säulen des maschinellen Lernens

Überwachtes Lernen (Supervised)



- **Mechanik:** Algorithmus lernt aus historischen, gelabelten Daten, um Zielwerte vorherzusagen (z. B. Umsatz, Klassifikationen).
- **Zweck:** Präzise Modellierung bekannter Zusammenhänge.

Unüberwachtes Lernen (Unsupervised)



- **Mechanik:** Algorithmus sucht ohne vordefinierte Labels eigenständig nach verborgenen Mustern und Strukturen.
- **Zweck:** Segmentierung, Dimensionsreduktion und Anomalieerkennung.

Bestärkendes Lernen (Reinforcement)



- **Mechanik:** Ein Agent lernt durch Interaktion mit einer Umgebung und iterative Belohnungs-/ Bestrafungsmechanismen.
- **Zweck:** Adaptive Entscheidungsfindung und dynamische Optimierung.

Der Diagnostik-Katalog: KI-Algorithmen im Überblick

Überwacht (Blau)

Lineare Regression

- Prognose kontinuierlicher Werte (z.B. Umsatz). Stärke: Interpretierbarkeit.

Logistische Regression

- Wahrscheinlichkeitsbasierte binäre Klassifikation (z.B. Krankheitsrisiko).

Decision Tree & Random Forest

- Regelbasierte Entscheidungsbäume / Ensemble-Methode zur Verringerung von Varianz.

Support Vector Machines (SVM)

- Findet optimale Trennlinien für Klassifikation im Merkmalsraum.

k-Nearest Neighbors (k-NN)

- Klassifiziert Instanzen basierend auf den k nächsten Nachbarn.

Neuronale Netze

- Komplexe Mustererkennung durch verbundene Neuronen-Schichten.

Naive Bayes

- Wahrscheinlichkeitsschätzung unter Annahme von Merkmalsunabhängigkeit.

Unüberwacht (Teal)

K-Means

- Gruppiert Daten durch Minimierung der mittleren Entfernung (Kundensegmentierung).

PCA

- Dimensionsreduktion durch Extraktion der wichtigsten Merkmale.

Apriori

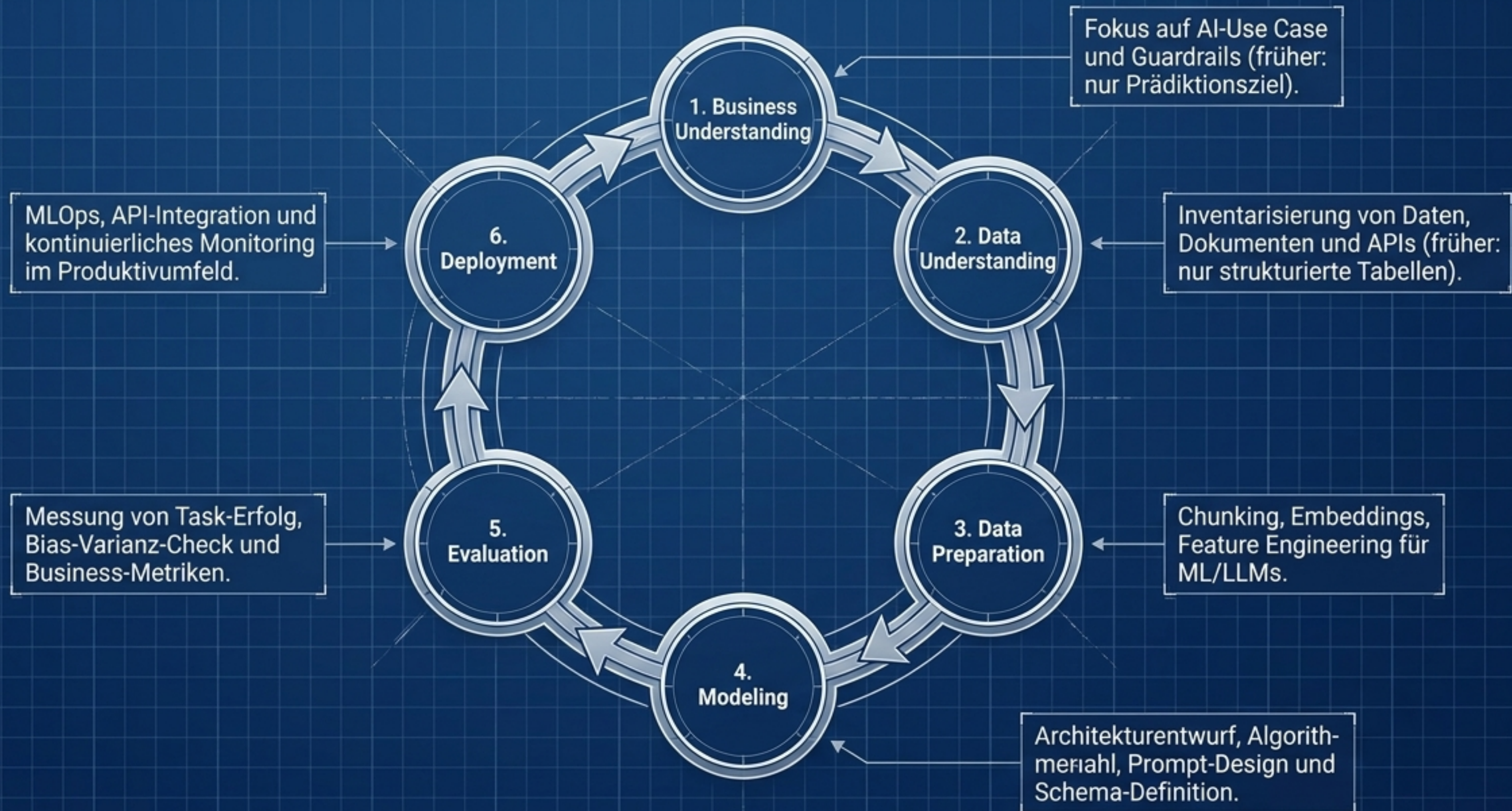
- Identifikation von Assoziationsregeln für häufig gemeinsame Itemsets.

Bestärkend (Orange)

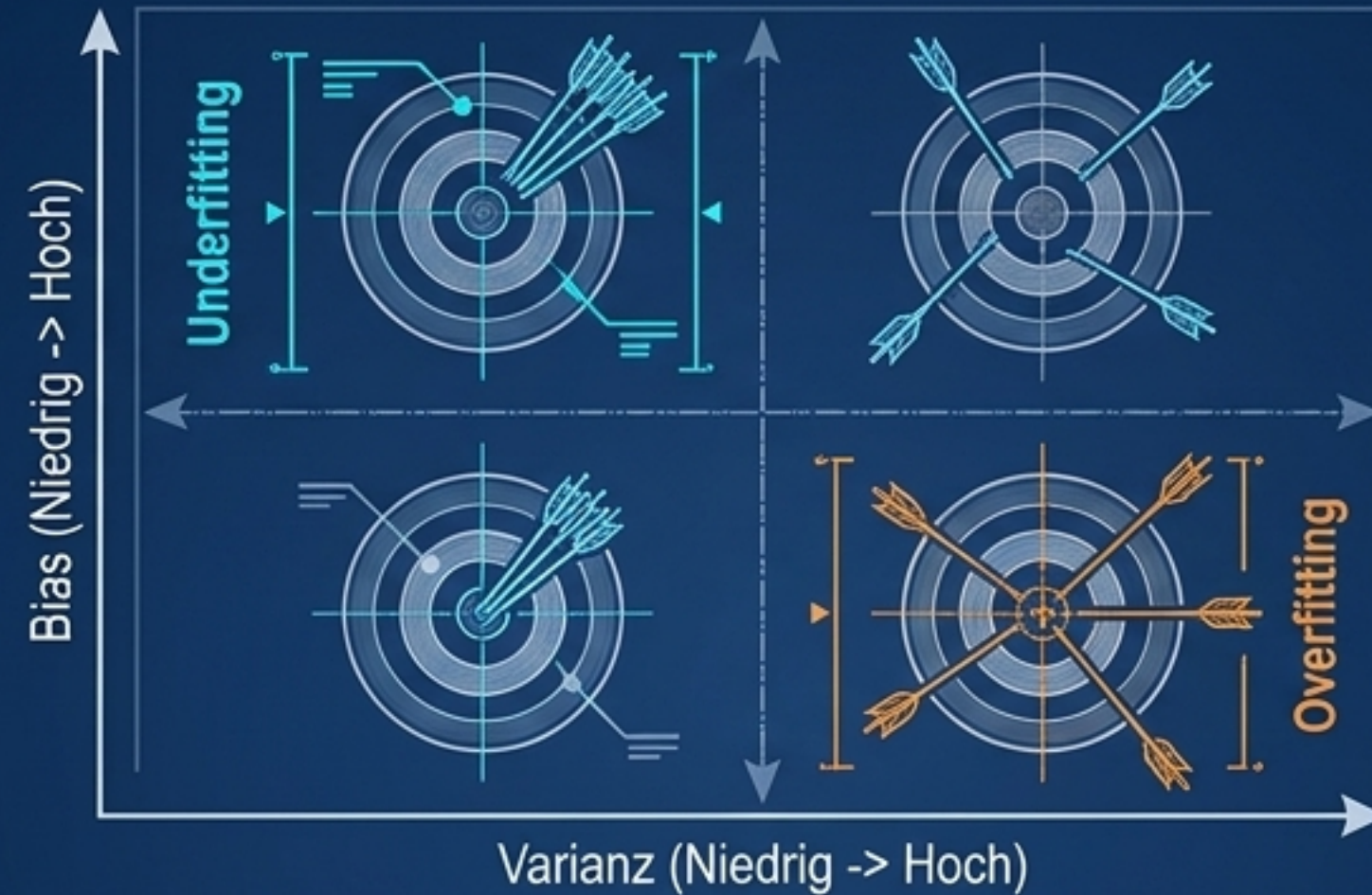
Deep-Q-Networks (DQN)

- Kombiniert Reinforcement Learning mit Deep Learning für komplexe Zustandsräume.

Das KI-Vorgehensmodell: CRISP-DM für moderne AI-Systeme



Modelldiagnostik: Das Bias-Varianz-Dilemma



Underfitting (High Bias, Low Variance)

Pfeile eng beieinander, aber weit weg vom Ziel. Das Modell ist zu simpel und erfasst die Datenstruktur nicht. (z.B. Lineare Regression auf nicht-lineare Daten).

Overfitting (Low Bias, High Variance)

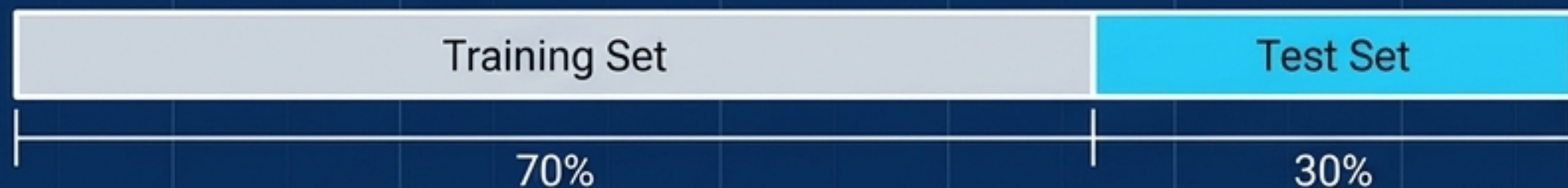
Pfeile treffen den Mittelpunkt im Durchschnitt, streuen aber extrem. Das Modell lernt Rauschen auswendig und scheitert an neuen Daten. (z.B. ungekürzte Decision Trees).

$$\text{Loss} = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

Die Kunst besteht in der Balance beider Fehlerquellen.

Absicherung der Generalisierung: Validierung und Resampling

Die Basis: Train/Test Split



Strikte Trennung, um das Modell auf unseen data zu evaluieren. Verhindert die Illusion einer perfekten Modellierung durch Overfitting.

Erweiterte Sicherheit: K-Fold Cross-Validation



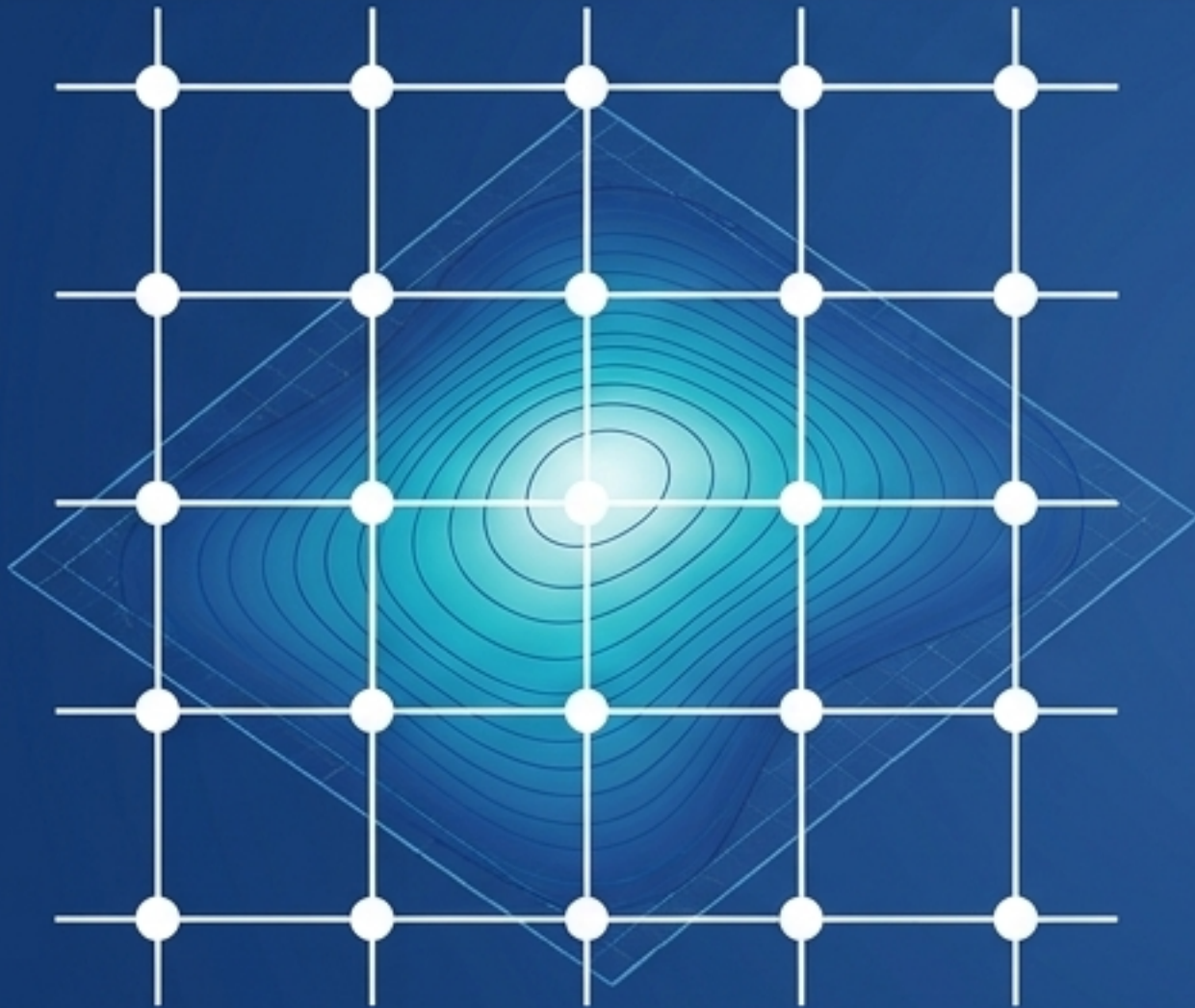
Der Datensatz wird in K (z.B. 5) gleich große Teile zerlegt.

Das Modell wird K-mal trainiert, wobei jeweils ein anderer Block zur Validierung dient.

Dies reduziert die Varianz der Performance-Schätzung erheblich.

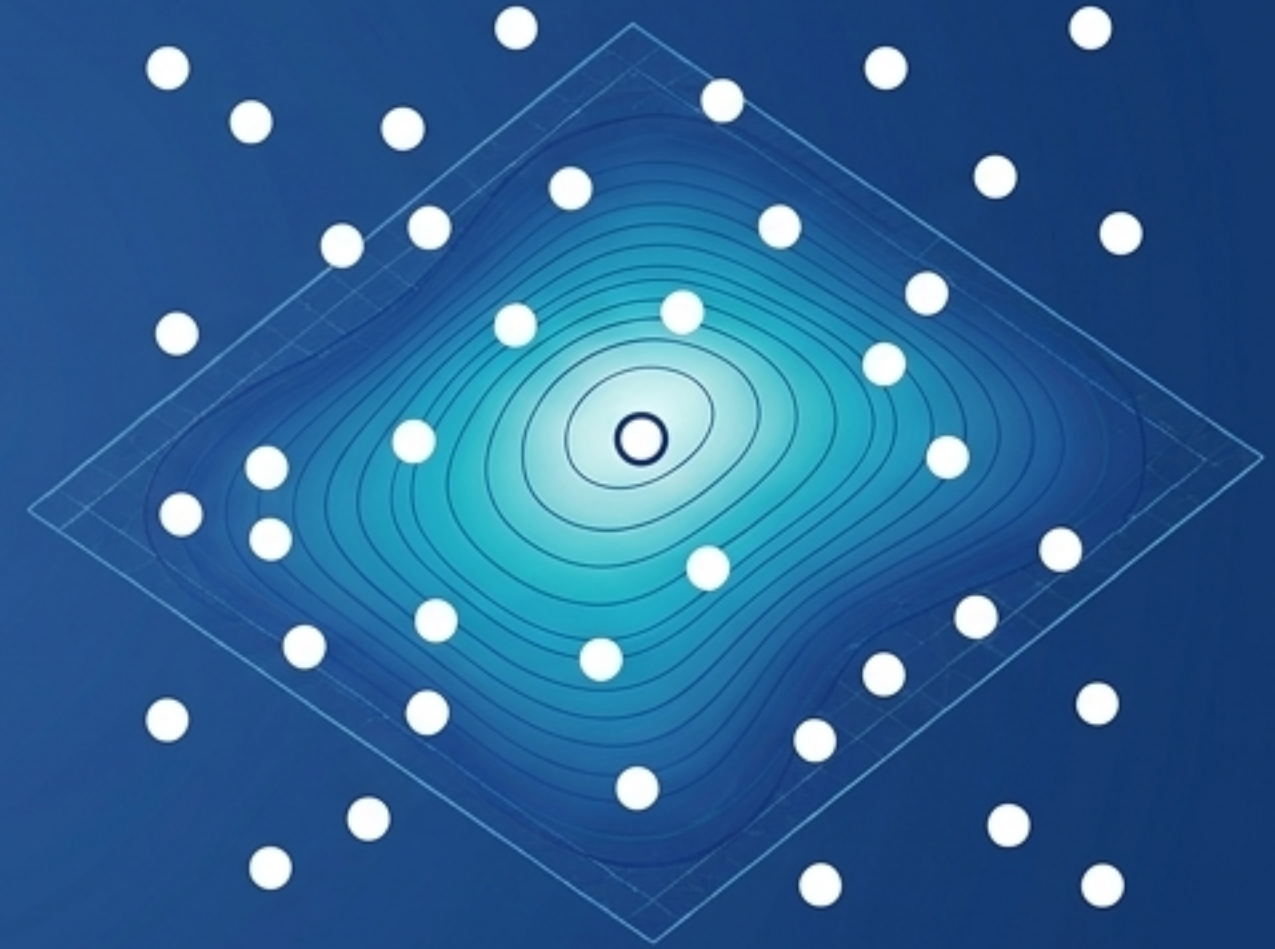
Optimierung des Modells: Parametertuning Strategien

Grid Search (Raster-Suche)



Definiert diskrete Werte (z.B. $C = [0.1, 1, 10]$) und testet jede mögliche Kombination. Sehr rechenintensiv und überseht oft das globale Optimum zwischen den Rasterpunkten.

Random Search (Zufallssuche)



Zieht Werte aus einer definierten Verteilung (z.B. loguniform). Effizienter in hochdimensionalen Räumen, da pro Suchlauf mehr unterschiedliche Werte evaluiert werden.

Business Szenario 1: Präzise Vorhersagen durch überwachtes Lernen

Use Case A: Regression zur Schätzung von Kennzahlen



Ziel:
Vorhersage von Fahrzeugverbrauch (Auto MPG) oder Immobilienwerten.

Evaluations-Metrik:
Mean Squared Error (MSE) / Negative MAE.

Use Case B: Klassifikation für Entscheidungsprozesse

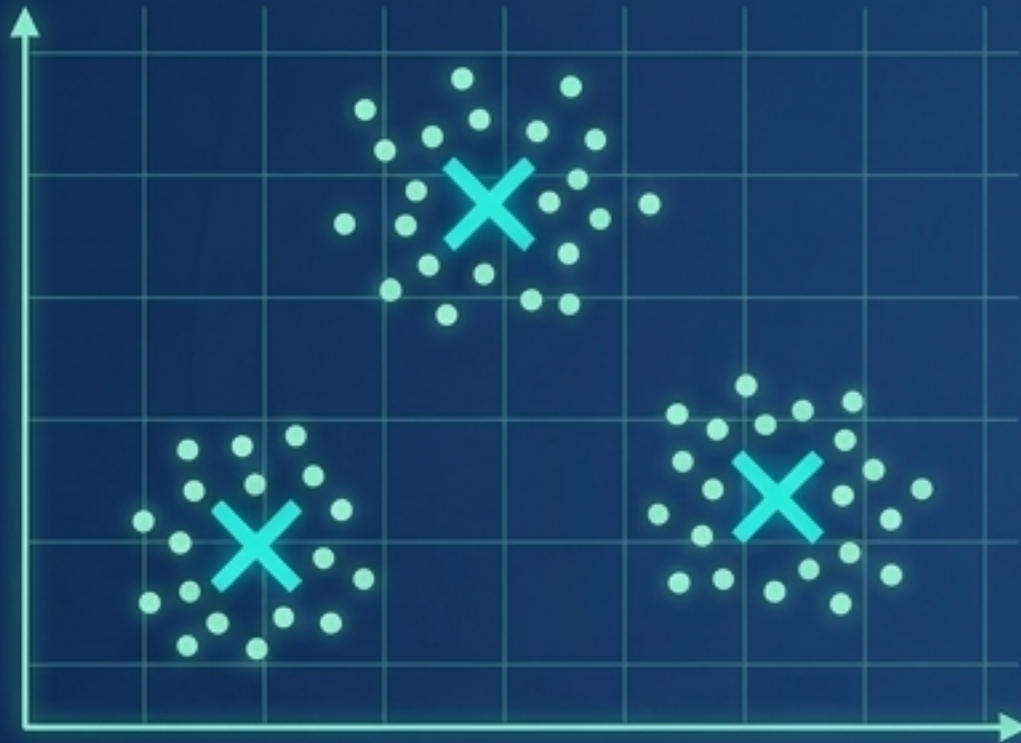


Ziel:
Medizinische Diagnosen (Diabetes) oder Kreditrisiko.

Evaluations-Metrik:
Precision, Recall, F1-Score.

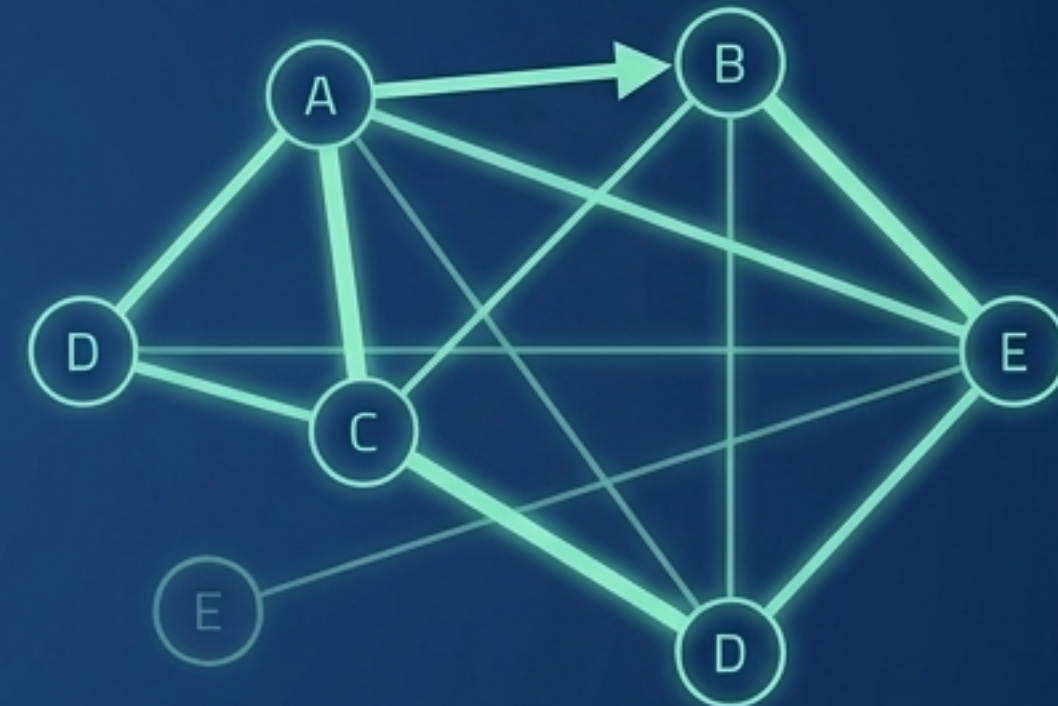
Business Szenario 2: Mustererkennung ohne Labels

Use Case C: Clustering zur Kundensegmentierung



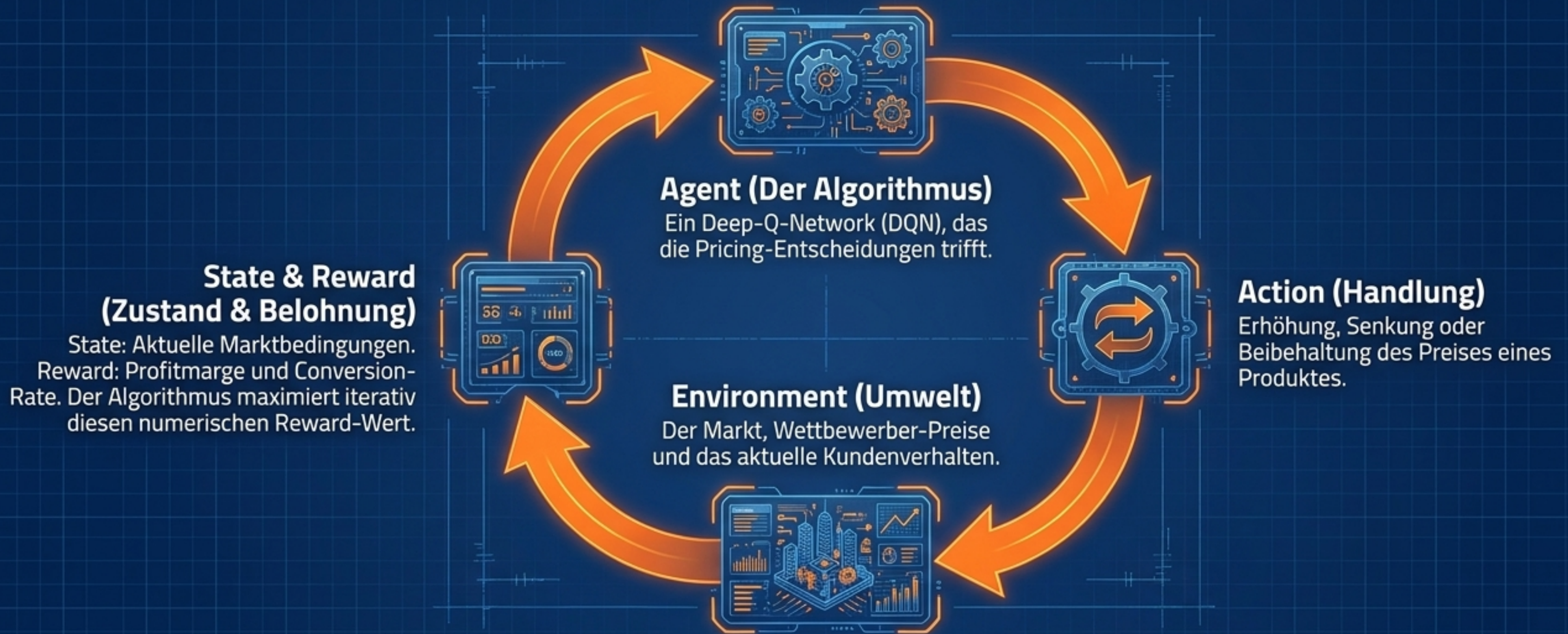
Ziel:	Zielgerichtetes Marketing in der Modebranche.
Algorithmus:	K-Means Clustering.
Mechanik:	Gruppert ähnliche Transaktionsmuster durch Minimierung der euklidischen Distanz.

Use Case D: Empfehlungsmaschinen für Cross-Selling



Ziel:	Warenkorbanalyse im Online-Handel.
Algorithmus:	Apriori-Algorithmus.
Metriken:	- Support: Häufigkeit des Itemsets im Datensatz. - Confidence: Wahrscheinlichkeit, dass Produkt B gekauft wird, wenn A im Korb ist. - Lift: Stärke der Assoziation (>1 = positive Korrelation).

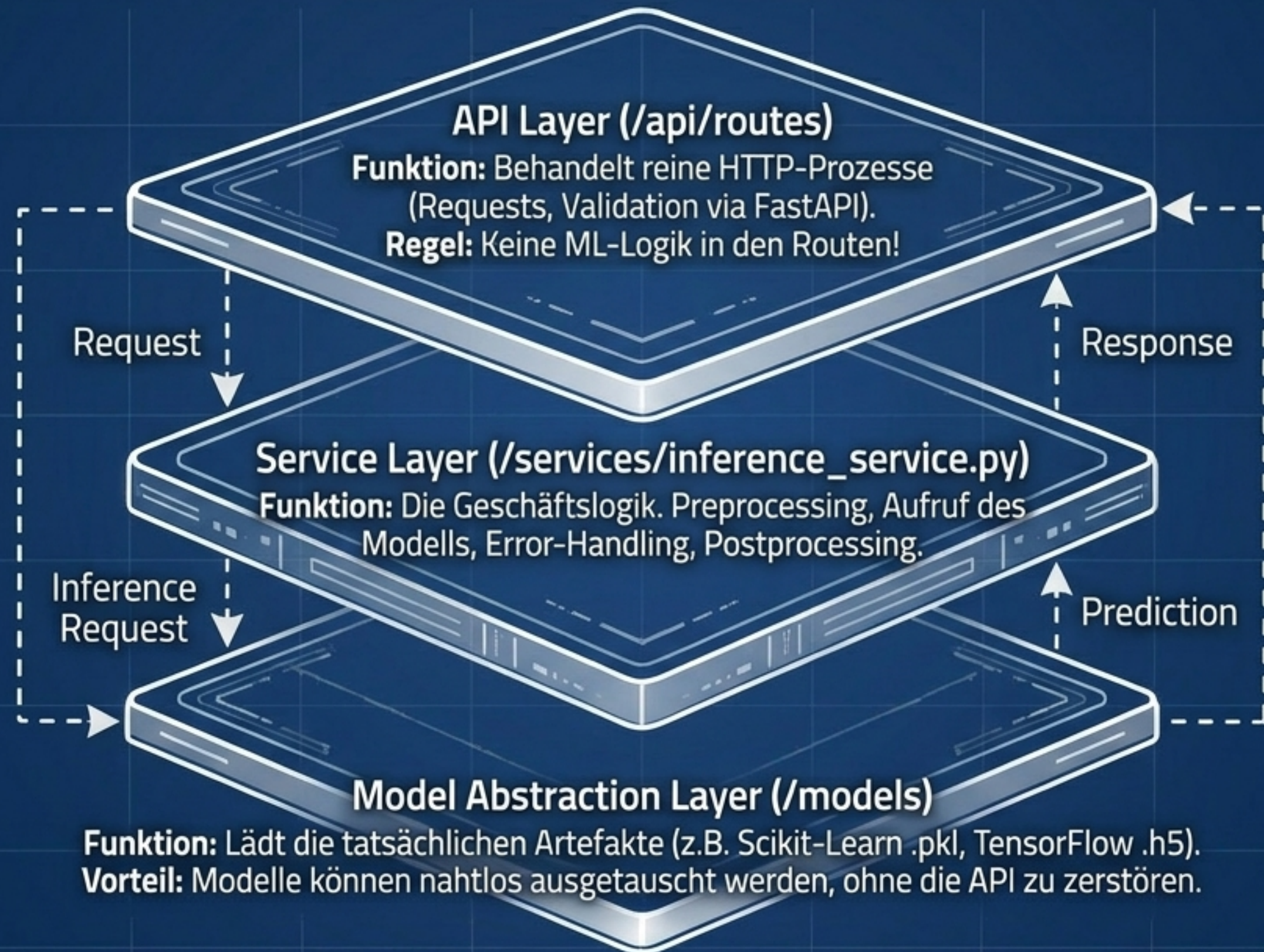
Business Szenario 3: Dynamic Pricing in volatilen Märkten



Fazit:

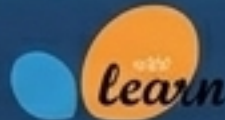
Optimal für komplexe, sich ständig ändernde Umgebungen, bei denen historische Daten allein für Vorhersagen nicht ausreichen.

MLOps-Architektur: Von der Theorie zur stabilen API



Praxisübungen: Der Python Tool-Stack

Scikit-Learn (Supervised Workflow)



```
X_train, X_test, y_train, y_test =  
train_test_split(X, y, test_size=0.2)
```

Datenspalten

```
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X_train)
```

Normalisierung

```
model = RandomForestClassifier(n_estimators=100)  
model.fit(X_train, y_train)  
predictions = model.predict(X_test)
```

Training

Inferenz

TensorFlow/Keras (Neuronale Netze)

```
model = Sequential([  
    Dense(64, activation='relu', input_shape=(10,)),  
    Dense(1, activation='linear')  
])
```

Architekturaufbau

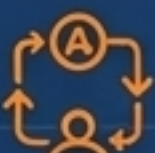
```
model.compile(optimizer='RMSPROP', loss='mse')
```

Konfiguration

```
model.fit(features, labels, epochs=100, batch_size=32)
```

Training

Synthese: Die "Business to Code" Matrix

Business Goal	ML Approach		Recommended Algorithm(s)	Evaluation Metric	Python Library
Kennzahlen prognostizieren	Überwacht (Regression)		Ridge, Neuronale Netze	MSE / MAE	Scikit-Learn, TF
Risiken klassifizieren	Überwacht (Klassifikation)		Random Forest, SVM	F1-Score	Scikit-Learn
Kunden segmentieren	Unüberwacht (Clustering)		K-Means	Silhouetten-Koeffizient	Scikit-Learn
Empfehlungen generieren	Unüberwacht (Assoziation)		Apriori	Lift, Confidence	mlxtend
Dynamische Preisgestaltung	Bestärkend (RL)		Deep-Q-Networks	Kumulierter Reward	PyTorch/TF

Praxis-Checkliste: Die MLOps Baseline

- | | |
|-----|--|
| [] | 1. Datenqualität gesichert? Fehlwerte (NaNs) imputiert und kategoriale Variablen encodiert? |
| [] | 2. Baseline etabliert? Ein simples Modell (z.B. Lineare Regression) trainiert, um einen Mindeststandard zu setzen? |
| [] | 3. Daten strikt getrennt? Train/Test-Split sauber durchgeführt, um Data-Leakage zu vermeiden? |
| [] | 4. Hyperparameter optimiert? Random Search oder Grid Search durchgeführt? |
| [] | 5. Bias-Varianz geprüft? Trainings- und Test-Fehler verglichen, um Overfitting auszuschließen? |
| [] | 6. Software-Architektur entkoppelt? ML-Logik sauber von der API-Schicht (z.B. FastAPI) getrennt? |

Vom Algorithmus zum funktionierenden KI-System

1.

1. Technologie ist nur das Werkzeug

Ob **Regression**, **K-Means** oder **Reinforcement Learning**: Der Algorithmus muss dem spezifischen Business Case folgen, nicht umgekehrt.

2.

2. Evaluation schlägt Intuition

Ein Modell ist nur so gut wie seine Fähigkeit zu **generalisieren**. Stichprobenbildung, **Resampling** und das Verständnis des **Bias-Varianz-Dilemmas** sind unerlässlich.

3.

3. **MLOps** ist der Schlüssel zur Produktion

Die besten Modelle scheitern ohne sauberes **Software Engineering**. Die Trennung von API, Geschäftslogik und **Modellabstraktion** garantiert langfristigen Erfolg.